VIVA PREPARATION GUIDE

Complete Code Explanation & Viva Q&A

"Making complex government policies simple and accessible for every citizen."

Language Python (Backend + AI)	**Frontend** HTML + React + JavaScript
AI Models TF-IDF + T5 Transformer	**NLP Tools** NLTK + spaCy
Database SQLite	**Framework** FastAPI

TABLE OF CONTENTS

1. Project Overview & Architecture

Python is the main language for ALL backend logic, AI processing, NLP, and database operations. The frontend (HTML + React) is only for displaying results in the browser — no AI logic lives there.

What the project does:

- User uploads a government policy PDF through the browser
- FastAPI backend receives it and saves it to disk
- The AI pipeline extracts text, cleans it, summarises it, finds keywords and entities
- Results are saved to SQLite and sent back as JSON to the browser
- React frontend renders Summary, Key Points, Keywords, Entities, Citizen Explanation

Architecture Diagram:

Layer	Technology	Language
Browser UI	React + HTML	JavaScript
REST API	FastAPI	Python
AI Pipeline	NLTK, spaCy, Transformers	Python
Database	SQLite (sqlite3)	Python
PDF Reader	PyPDF2	Python

Project File Structure:

`backend/main.py`	FastAPI web server — 3 API endpoints
`backend/database.py`	SQLite setup — save and fetch documents
`modules/pdf_extractor.py`	Extract text from PDF using PyPDF2
`modules/preprocessing.py`	NLTK — tokenize, remove stopwords, lemmatize
`modules/summarizer.py`	TF-IDF extractive + T5 abstractive summarization
`modules/keywords_ner.py`	Keyword frequency + spaCy Named Entity Recognition
`modules/simplifier.py`	Flan-T5 — citizen-friendly plain language explanation
`modules/pipeline.py`	Orchestrates all modules in sequence
`frontend/index.html`	React UI — hero, upload, results, history

2. backend/main.py — FastAPI Web Server

backend/main.py Language: Python | Library: FastAPI, uvicorn

Purpose:

This is the entry point of the entire application. Running `python main.py` starts a web server on port 8000. It acts as the bridge between the browser and all the AI modules.

Startup Event:

When the server starts, `@app.on_event("startup")` runs two things: (1) `init_db()` creates the SQLite database table if it does not exist, and (2) `PolicyPipeline()` loads all AI models into memory. Loading models once at startup means the first request is fast — models are never reloaded.

Three API Endpoints:

Endpoint	Method	What it does
`/upload-policy`	POST	Receives PDF file, saves to disk, returns unique document_id
`/summarize-policy`	POST	Runs full AI pipeline on saved PDF, returns results as JSON
`/history`	GET	Returns list of all previously analysed documents from database

Key concept — UUID:

`uuid.uuid4()` generates a random unique ID like `f47ac10b-58cc-4372-a567-0e02b2c3d479` for every uploaded file. This prevents filename collisions if multiple users upload files with the same name.

Error handling:

`HTTPException` is used to return meaningful error messages. For example, if a non-PDF file is uploaded, the server immediately returns HTTP 400 with message "Only PDF files accepted." before any processing happens.

3. backend/database.py — SQLite Storage

backend/database.py Language: Python | Library: sqlite3 (built into Python)

Why SQLite:

SQLite is a file-based database requiring zero installation. The entire database is one file: `data/policy.db`. No separate database server is needed, making the project easy to run locally on any laptop.

Database Table — documents:

Column	Type	What it stores
`id`	TEXT (Primary Key)	Unique UUID for each document
`filename`	TEXT	Original PDF filename
`summary`	TEXT	AI-generated summary paragraph
`key_points`	TEXT (JSON)	List of 5 key sentences stored as JSON string
`keywords`	TEXT (JSON)	List of 10 keywords stored as JSON string
`entities`	TEXT (JSON)	List of named entities with labels, as JSON string
`citizen_explanation`	TEXT	Plain language explanation for citizens
`word_count`	INTEGER	Total words in the original document
`processing_time`	REAL	Seconds taken to process
`created_at`	TEXT	ISO timestamp when processed

Why JSON for lists:

SQLite does not natively support arrays or lists. Key points, keywords, and entities are Python lists, so they are serialised to JSON strings with `json.dumps()` before saving, and parsed back with `json.loads()` when reading.

4. modules/pdf_extractor.py — PDF Reading

modules/pdf_extractor.py Language: Python | Library: PyPDF2

What it does:

PDFs are not plain text — they store content in a binary compressed format. PyPDF2 decodes this format and extracts the raw text from every page. All page texts are joined into a single string and returned along with the page count and word count.

Core code logic:

```
reader = PyPDF2.PdfReader(f)
for page in reader.pages:
text += page.extract_text()
```

Important limitation:

If the PDF is a scanned image (a photograph of a document), PyPDF2 returns empty text because there is no text layer — only pixels. The pipeline raises a clear ValueError: "No text could be extracted. The PDF may be image-only."

5. modules/preprocessing.py — NLP Text Cleaning

modules/preprocessing.py Language: Python | Library: NLTK (Natural Language Toolkit)

Three NLP preprocessing steps (as shown in the technical diagram):

Step 1 — Tokenization

Tokenization breaks text into individual units called tokens. sent_tokenize() splits by sentences. word_tokenize() splits by words. Example: "The policy shall fund programs." becomes ["The", "policy", "shall", "fund", "programs", "."]

```
sentences = sent_tokenize(cleaned) tokens = word_tokenize(cleaned.lower())
```

Step 2 — Stopword Removal

Stopwords are common words like "the", "is", "at", "which", "and" that carry no meaningful information. NLTK provides a list of 179 English stopwords. Filtering them out reduces noise so the AI focuses only on meaningful content words.

```
STOP = set(stopwords.words("english")) filtered = [t for t in tokens if t not in STOP]
```

Step 3 — Lemmatization

Lemmatization reduces words to their dictionary root form. "running" becomes "run", "policies" becomes "policy", "established" becomes "establish". This ensures different forms of the same word are treated identically by the AI models.

```
LEM = WordNetLemmatizer() lemmatized = [LEM.lemmatize(t) for t in filtered]
```

Text cleaning (regex):

Before tokenizing, raw PDF text is cleaned using regular expressions: non-printable characters are removed, excessive blank lines are normalised, and standalone page numbers (common PDF artifacts) are stripped.

6. modules/summarizer.py — Summarization Engine

modules/summarizer.py	Language: Python | Library: math, collections, HuggingFace Transformers (T5)

Two summarization methods:

Method 1 (TF-IDF) runs first to reduce a 4,000-word document to ~300 words. Method 2 (T5) then rewrites those 300 words into a clean 150-200 word summary. This two-stage approach is more efficient than feeding the full document directly to T5.

Method 1 — Extractive Summarization using TF-IDF

TF-IDF stands for **Term Frequency — Inverse Document Frequency**. It scores each sentence based on how important its words are.

- TF (Term Frequency): How often a word appears in a sentence divided by total words. If "policy" appears 3 times in a 30-word sentence, TF = 3/30 = 0.1
- IDF (Inverse Document Frequency): How rare the word is across ALL sentences. "the" appears everywhere so its IDF is very low. "allocate" is rare so its IDF is high.
- TF-IDF score = TF × IDF. Sentences with high scores contain important, specific terms.
- The top-scoring sentences are selected and returned in their original order from the document. This is called EXTRACTIVE because you extract existing sentences — no new text is created.

```
score = sum( (tf[w]/len(words)) * math.log((N+1)/(doc_freq[w]+1)) for w in tf )
```

Method 2 — Abstractive Summarization using T5

T5 (Text-to-Text Transfer Transformer) is a deep learning language model developed by Google. Unlike TF-IDF, T5 does not select existing sentences — it *generates* entirely new sentences that capture the meaning, just like a human would paraphrase a document. `device=-1` forces CPU usage so no GPU is needed.

```
_t5 = pipeline("summarization", model="google/flan-t5-base", device=-1)
result = _t5(text, max_length=200, min_length=60, do_sample=False)
```

7. modules/keywords_ner.py — Keywords & Named Entities

modules/keywords_ner.py Language: Python | Library: collections.Counter + spaCy

Keyword Extraction — Frequency based:

After removing stopwords, the most frequently occurring words are counted using Python's `collections.Counter`. The top 10 words are returned as keywords. This works well for policy documents because important terms like "district", "funding", "ministry" are naturally repeated throughout.

```
return [w for w, _ in Counter(filtered).most_common(10)]
```

Named Entity Recognition (NER) — spaCy:

spaCy's `en_core_web_sm` model is a pre-trained neural network that identifies and classifies proper nouns in text. It has learned from millions of documents to recognise entity patterns.

Label	Meaning	Example from policy
ORG	Organisation	Ministry of Electronics and IT
DATE	Date or time period	March 2025, financial year 2024
MONEY	Budget or amount	■2,400 crore, USD 500 million
LAW	Law or policy name	National Digital Literacy Policy
GPE	Location / Government	India, Maharashtra, New Delhi
PERSON	Named person	Prime Minister, Secretary

```
doc = _nlp(text[:6000])
for ent in doc.ents:
entities.append({"text": ent.text, "label": ent.label_})
```

8. modules/simplifier.py — Citizen-Friendly Explanation

modules/simplifier.py Language: Python | Library: HuggingFace Transformers (Flan-T5)

What is Flan-T5:

Flan-T5 is an instruction-tuned version of T5. "Instruction-tuned" means it was trained on examples of instructions paired with expected responses. This allows it to follow natural language commands like "Explain in simple words." The model adapts its output tone and vocabulary based on the instruction prefix.

The prompt:

```
"Explain this government policy in simple words for ordinary citizens."
"Use short sentences. Avoid jargon:" + summary[:800]
```

Fallback — Rule-based simplification:

If the model fails or produces output that is too short, a dictionary-based fallback replaces complex legal terms with simpler equivalents:

Complex term	Plain equivalent
`pursuant to`	according to
`notwithstanding`	despite
`mandate`	require
`provisions`	rules
`legislation`	law
`implementation`	putting into practice
`stakeholders`	people involved

9. modules/pipeline.py — The Orchestrator

modules/pipeline.py Language: Python | Library: Calls all other modules

Role:

pipeline.py is the conductor. It imports every other module and calls them in the correct sequence. The main.py backend only needs to call `pipeline.process(pdf_path)` and receives one complete result dictionary containing everything.

Processing sequence:

#	Step	Module	What happens
1	**PDF Extraction**	`pdf_extractor.py`	PyPDF2 reads all pages → raw text string
2	**Preprocessing**	`preprocessing.py`	Tokenize → remove stopwords → lemmatize
3	**TF-IDF Summary**	`summarizer.py`	Score sentences → select top 6 → join
4	**T5 Summary**	`summarizer.py`	Neural model rewrites TF-IDF output into clean summary
5	**Key Points**	`summarizer.py`	Top 5 TF-IDF scored sentences from full document
6	**Keywords**	`keywords_ner.py`	Top 10 most frequent meaningful words
7	**NER**	`keywords_ner.py`	spaCy identifies organisations, dates, laws, locations
8	**Citizen Brief**	`simplifier.py`	Flan-T5 rewrites summary in plain everyday language

10. Frontend — React & JavaScript

frontend/index.html Language: HTML + JavaScript (React) | Library: React 18, Babel

What React does in this project:

- Displays the hero landing page with site name and tagline
- Handles drag-and-drop PDF upload
- Calls the FastAPI backend using the browser fetch() API
- Shows real-time processing progress with step-by-step animation
- Renders all result sections: Summary, Key Points, Keywords, Entities, Citizen Explanation
- Maintains History page showing previously analysed documents

How the frontend communicates with the backend:

React uses the browser's built-in `fetch()` function to make HTTP requests to the Python FastAPI server. For example, when the user clicks "Analyse Document":

```
const up = await fetch("http://localhost:8000/upload-policy", { method:"POST",
body:formData })
```

```
const res = await fetch("http://localhost:8000/summarize-policy?document_id="+id, {
method:"POST" })
```

Two-page scroll layout:

The home page uses CSS scroll — the first "page" is a full-viewport hero screen (100vh height) with the site name, tagline, and a bouncing scroll arrow. When the user scrolls down, the PDF upload section is revealed below it. This creates the effect of two distinct screens within a single HTML page.

11. Viva Q&A; — Key Questions & Answers

Q1. What is the main programming language used in this project and why?

A: Python is the main language for the entire backend — web server, AI processing, NLP, and database storage. Python was chosen because it has the richest ecosystem of AI and data science libraries: NLTK, spaCy, HuggingFace Transformers, FastAPI, and sqlite3 are all Python packages. The only non-Python part is the frontend, which uses HTML and JavaScript — that is only for displaying results in the browser.

Q2. What is the difference between extractive and abstractive summarization?

A: Extractive summarization selects and returns the most important existing sentences from the document using TF-IDF scoring — no new words are generated. Abstractive summarization uses a neural network (T5) to generate entirely new sentences that capture the meaning, like a human paraphrasing. This project uses both in sequence: TF-IDF reduces a long document to about 300 words, then T5 rewrites those into a clean 150-word summary.

Q3. Why do you use TF-IDF before T5? Why not just use T5 directly?

A: T5 has a maximum token limit — it cannot process very long documents in one pass. A 50-page policy document can be 15,000+ words. TF-IDF first selects the 6 most important sentences (~300 words), which T5 can comfortably process. This two-stage approach also produces better results because T5 receives a pre-filtered, coherent input rather than raw noisy text.

Q4. What is NER and why is it useful for policy documents?

A: NER stands for Named Entity Recognition. It is an NLP task where a pre-trained model identifies and classifies proper nouns in text — organisations, dates, amounts, laws, and locations. For government policy documents this is especially useful because policies are filled with ministry names, budget figures, deadline dates, and legal references. Automatically surfacing these gives citizens an instant snapshot of who is involved, when things happen, and how much money is allocated.

Q5. Why is FastAPI used instead of Flask?

A: FastAPI is faster than Flask, supports asynchronous operations natively, and automatically generates interactive API documentation at /docs. It uses Python type hints for automatic request validation — invalid inputs like non-PDF files are rejected before they reach the pipeline. FastAPI also uses uvicorn as its ASGI server, which handles concurrent requests more efficiently than Flask's development server.

Q6. What does device=-1 mean in the Transformers pipeline?

A: In HuggingFace Transformers, device=-1 means use CPU instead of GPU. Device 0 would mean use the first GPU (CUDA). Since this project is designed to run on a normal laptop without a graphics card, device=-1 forces all inference to run on the CPU. This makes the project accessible to anyone without expensive hardware.

Q7. Why SQLite and not MySQL or PostgreSQL?

A: SQLite requires zero installation and zero configuration — the entire database is a single .db file stored in the project folder. It is built into Python's standard library so no additional installation is needed. For a project of this scale — one user, occasional reads and writes — SQLite is more than sufficient. MySQL or PostgreSQL would require a separate database server, adding unnecessary complexity for a local project.

Q8. What happens if the uploaded PDF is a scanned image?

A: PyPDF2 can only extract text if the PDF has a digital text layer. A scanned PDF is essentially a photograph — it contains only pixels, not text characters. In that case extract_text_from_pdf() returns an empty string. The pipeline detects this and raises a ValueError with the message "No text could be extracted. The PDF may be image-only." This error is caught in main.py and returned to the frontend as HTTP 500 with a clear, user-readable message.

Q9. What are stopwords and why are they removed?

A: Stopwords are extremely common English words like "the", "is", "at", "which", "and", "a", "of" that appear in almost every sentence but carry no meaningful information. NLTK provides a list of 179 English stopwords. Removing them reduces the vocabulary size and ensures that TF-IDF, keyword extraction, and NER models focus only on meaningful content words rather than being dominated by function words.

Q10. What is lemmatization and how is it different from stemming?

A: Lemmatization reduces words to their dictionary root form (lemma) using vocabulary and morphological analysis. "running" becomes "run", "policies" becomes "policy". Stemming is a simpler, cruder approach that just chops off suffixes: "running" becomes "runn", "policies" becomes "polici" — which are not real words. Lemmatization is preferred because it produces actual dictionary words, making downstream processing more accurate.

Q11. What is the role of uuid.uuid4() in the upload endpoint?

A: uuid.uuid4() generates a random universally unique identifier — a 128-bit number expressed as a string like f47ac10b-58cc-4372-a567-0e02b2c3d479. Each uploaded PDF is saved with this UUID as its filename. This prevents collisions — if two users upload files both named "policy.pdf", they get separate IDs and separate saved files. The UUID also acts as the document's primary key in the SQLite database.

Q12. How does the frontend know when the AI has finished processing?

A: The frontend sends an upload request first (fast), then sends a summarize request. The summarize endpoint is synchronous — it blocks and waits for the entire AI pipeline to finish before returning the JSON response. While waiting, the frontend plays a step-by-step animation using JavaScript setTimeout to simulate progress. When the fetch() promise finally resolves with the result, the animation stops and the results page is shown with the real data.

Quick Reference — Libraries & Their Roles

Library	Language	Role in Project
`FastAPI`	Python	Web framework — handles HTTP requests, defines API endpoints
`uvicorn`	Python	ASGI server — actually runs the FastAPI application
`PyPDF2`	Python	Reads PDF files and extracts text from each page
`NLTK`	Python	Tokenization, stopword removal, lemmatization (NLP preprocessing)
`spaCy`	Python	Named Entity Recognition — identifies orgs, dates, laws, locations
`Transformers`	Python	Loads T5 and Flan-T5 models for summarization and simplification
`torch`	Python	Deep learning backend for Transformers (CPU mode, device=-1)
`sqlite3`	Python	Built-in database — stores all results without extra installation
`collections`	Python	Counter class used for keyword frequency counting
`math`	Python	log() function used in TF-IDF score calculation
`uuid`	Python	uuid4() generates unique IDs for each uploaded document
`re`	Python	Regular expressions for cleaning PDF text artifacts
`React 18`	JavaScript	Frontend UI library — renders the user interface in the browser
`fetch()`	JavaScript	Browser API used to make HTTP calls to the FastAPI backend